Chapter 3

The Processor

From Performance to Instructions to Processor

Chapter 1
Computer Performance

- How do we measure computer performance?
- CPU time is determined by
  - Instruction count
  - CPI (cycles per instruction)
  - Clock cycle time
- Understand how fast a computer can be

CPU time = Instruction count × CPI × Cycle time

- Determined by ISA and compiler
- Determined by CPU hardware
- Determined by CPU hardware

How fast can a computer be?

Chapter 2
Instructions and MIPS

- Learn the Instruction Set Architecture (ISA)
- Know how an instruction represents a computation
- Study MIPS instructions and their formats
- Understand what operations the CPU needs to execute

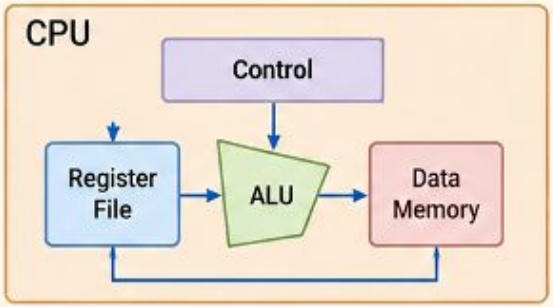
Examples of MIPS instructions

```
add   $t0, $t1, $t2     # arithmetic
lw    $t0, 0($t1)       # load from memory
sw    $t0, 0($t1)       # store to memory
beq   $t0, $t1, label   # branch
j     label             # jump
```

What should the CPU execute?

Chapter 3
The Processor

- **How does the processor actually execute an instruction?**
- What hardware components are needed inside the CPU?
- How does the instruction flow through these components?
- How does the hardware know what operation to perform for each instruction?



How does the CPU execute an instruction?

Instruction Execution

- PC → ινστρυχτιον μεμορψ, φετχη ινστρυχτιον
- Register numbers → ρεγιστερ φιλε, ρεαδ ρεγιστερσ
- Depending on instruction class
  - Use Arithmetic Logic Unit (ALU) to calculate
    - Arithmetic result
    - Memory address for load/store
    - Branch target address
  - Access data memory for load/store
  - PC ← target address or PC + 4

Instruction Execution

- PC → ινστρυχτιον μεμορψ, φετχη ινστρυχτιον
- Register numbers → ρεγιστερ φιλε, ρεαδ ρεγιστερσ
- Depending on instruction class

operations. Put them in the correct order.

A. Use the ALU for the required calculation

B. Fetch the instruction from Instruction Memory

C. Update the PC

D. Read the required registers from the Register File

- PC ← target address or PC + 4

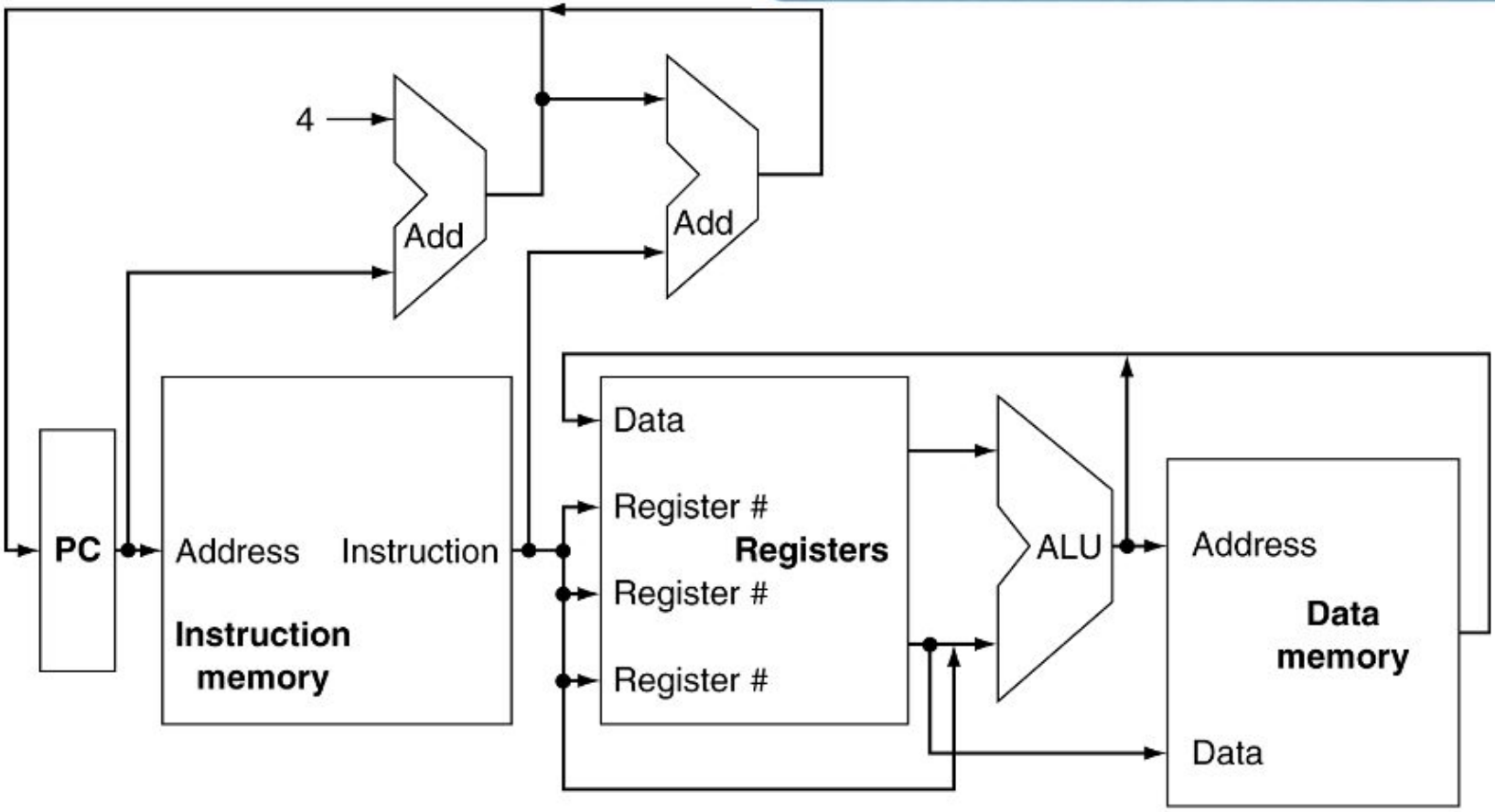
A1. B, D, A, C

CPU Overview

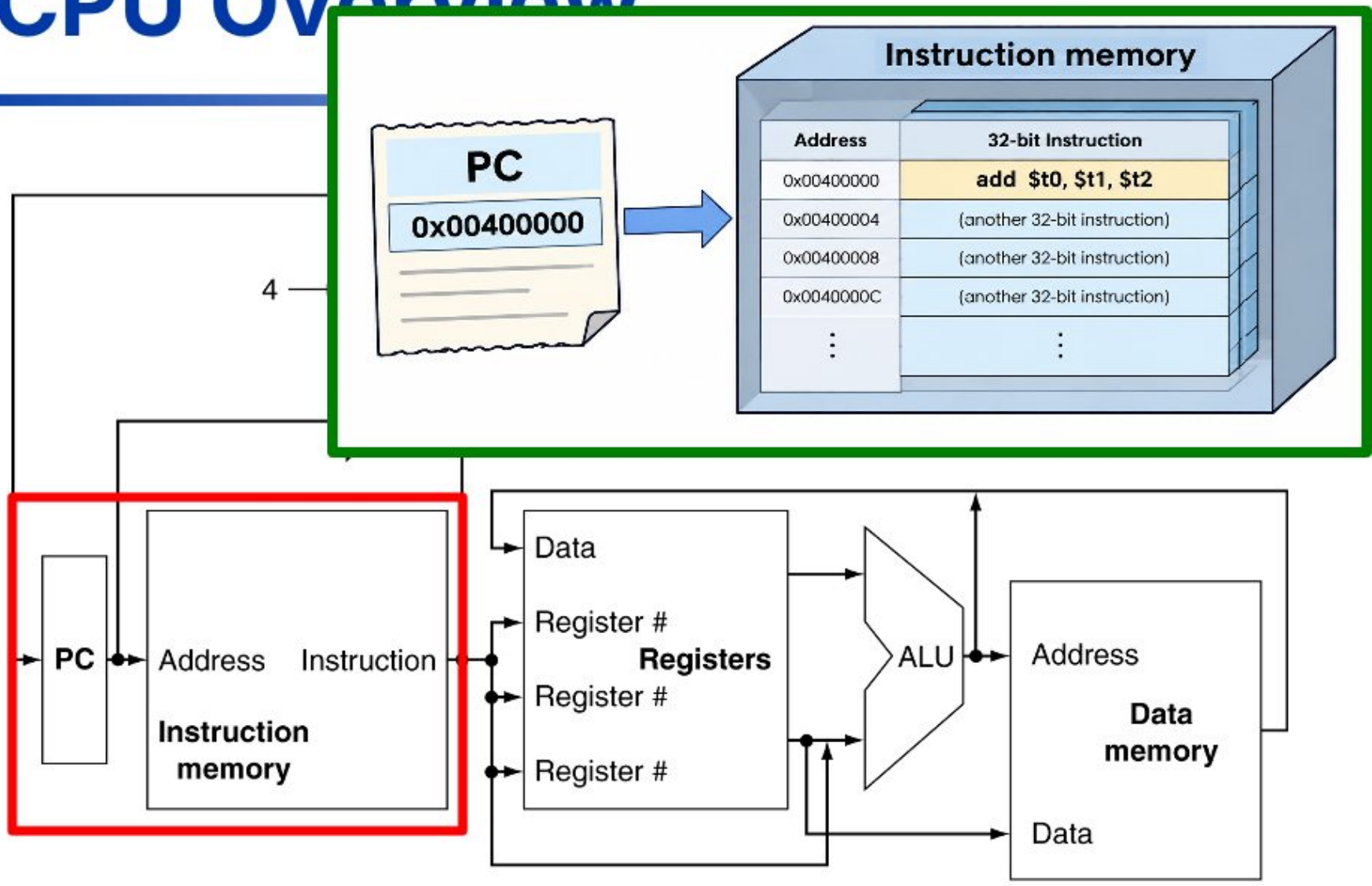
Example Instruction

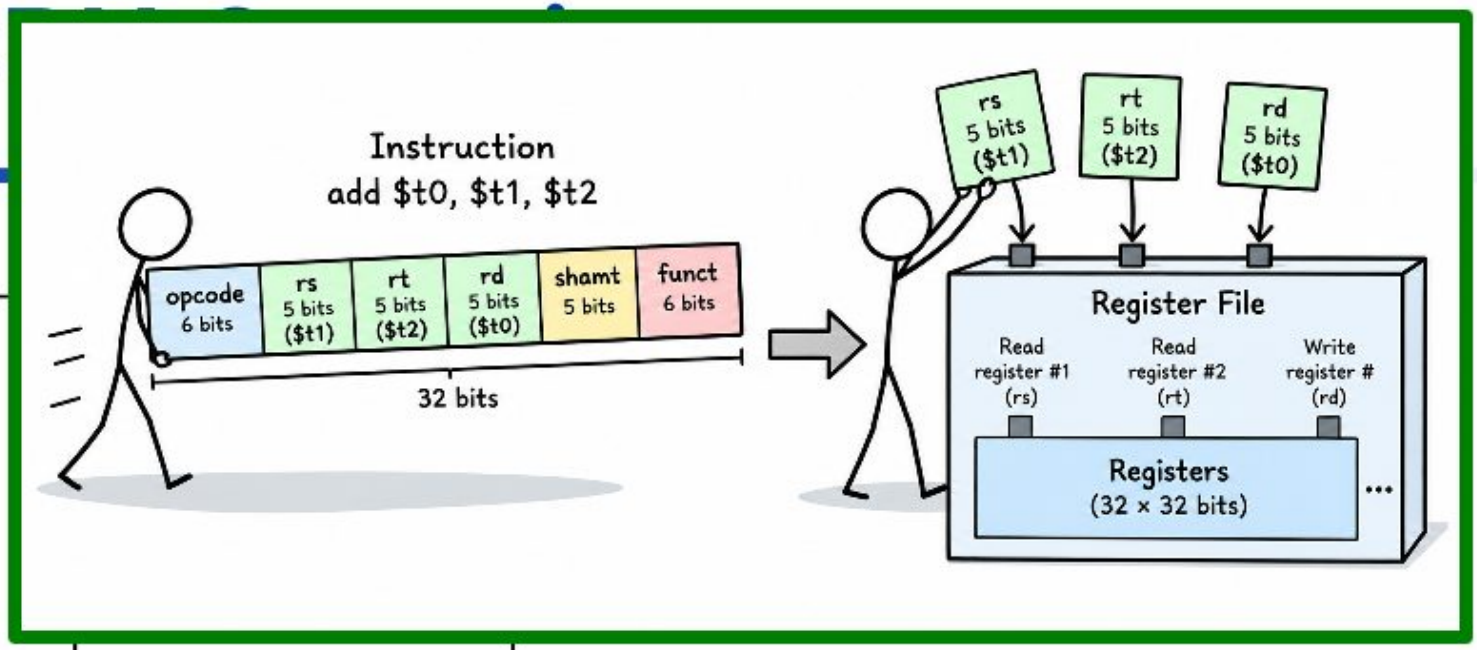
add $t0, $t1, $t2

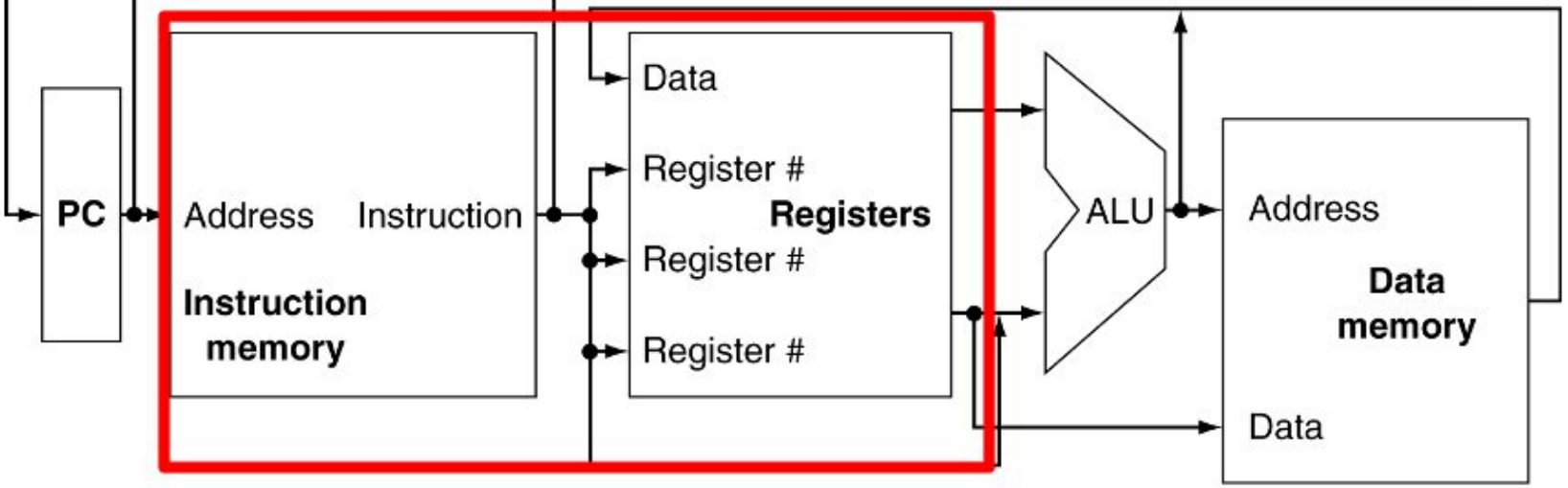
31–26	25–21	20–16	15–11	10–6	5–0
opcode 000000	**rs** 01001 ($t1)	**rt** 01010 ($t2)	**rd** 01000 ($t0)	**shamt** 00000	**funct** 100000 (add)

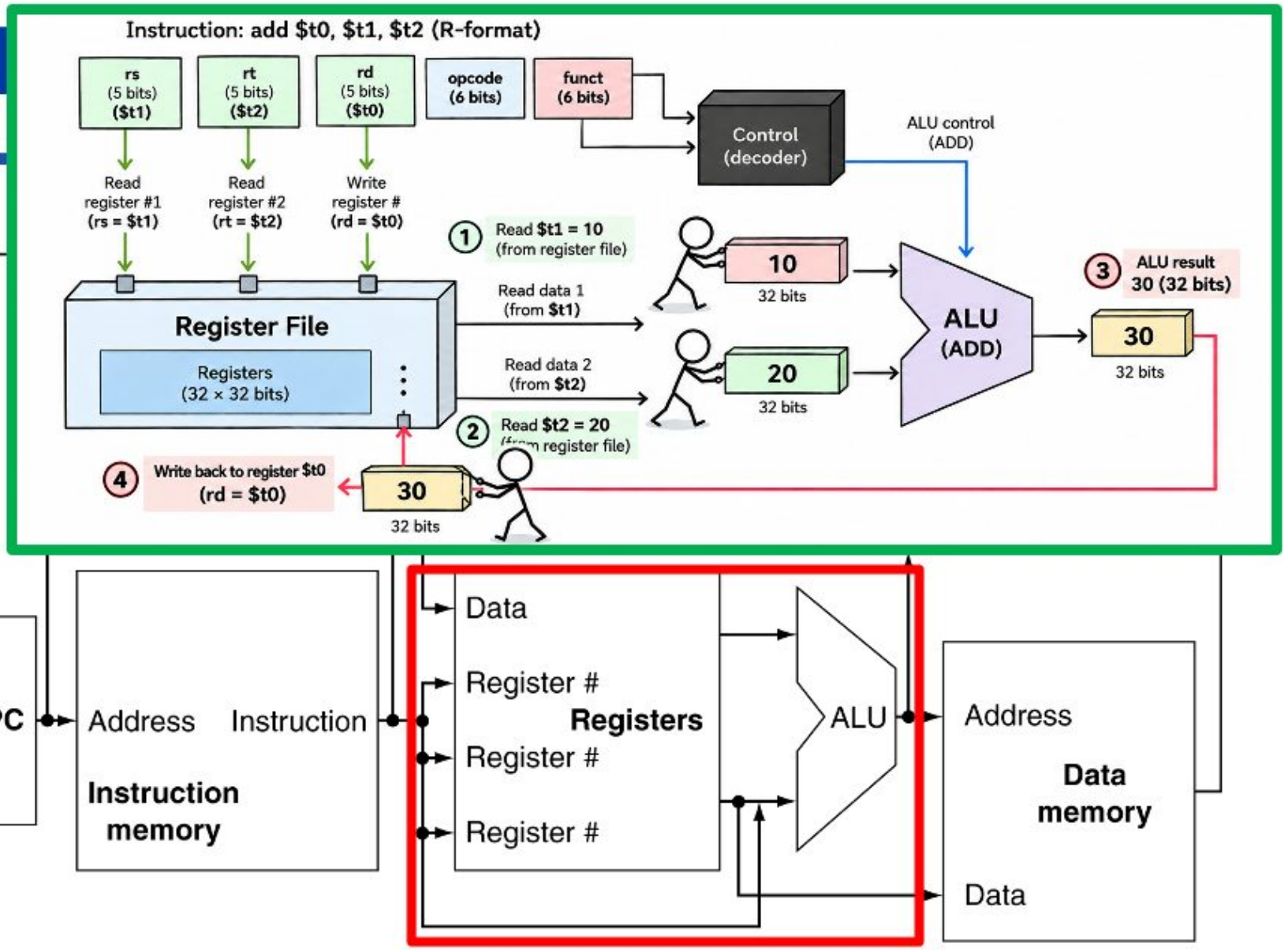


CPU Overview

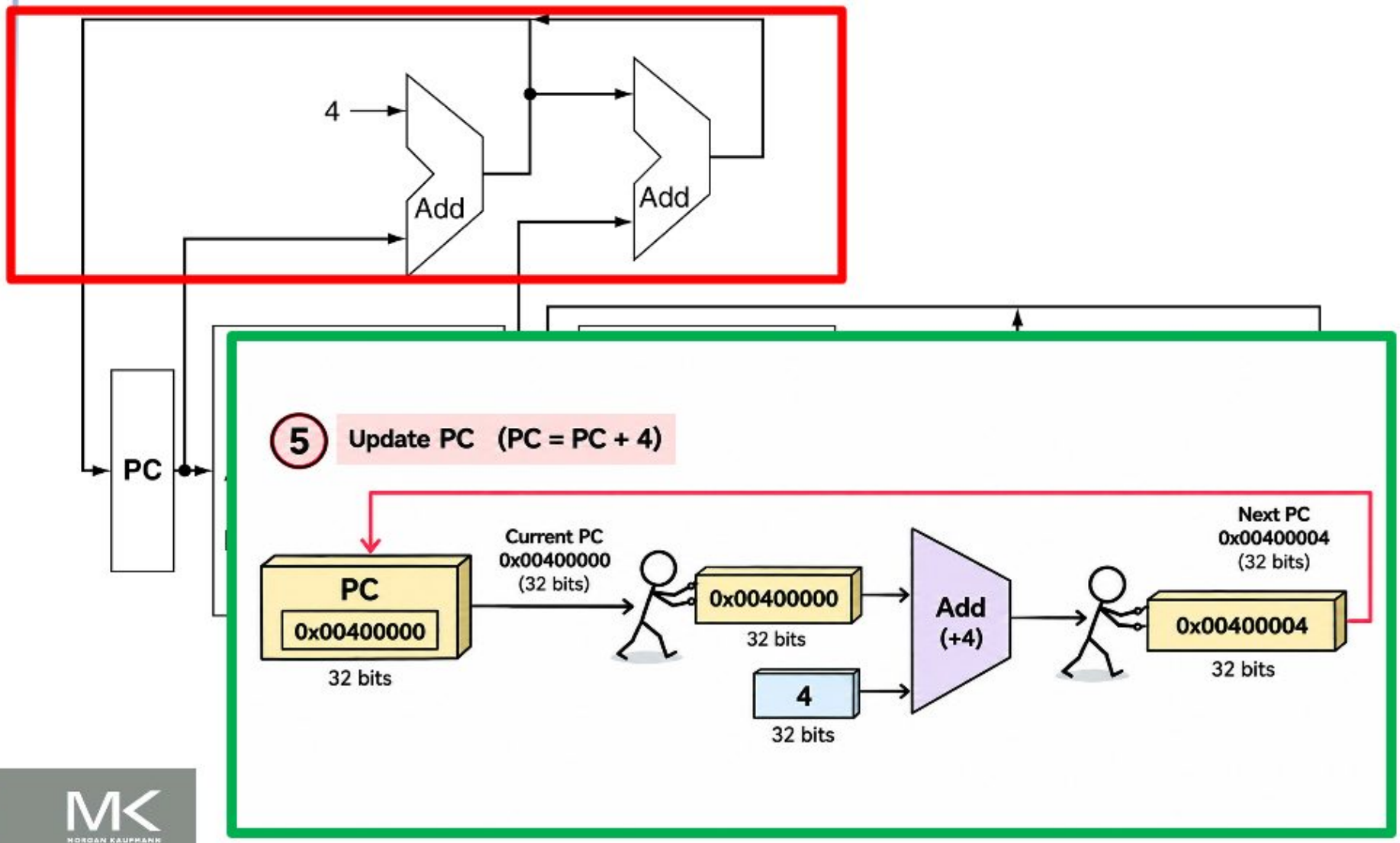



Instruction
add $t0, $t1, $t2
opcode
6 bits
rs
5 bits
($t1)
rt
5 bits
($t2)
rd
5 bits
($t0)
shamt
5 bits
funct
6 bits
32 bits
rs
5 bits
($t1)
rt
5 bits
($t2)
rd
5 bits
($t0)
Register File
Read
register #1
(rs)
Read
register #2
(rt)
Write
register #
(rd)
Registers
(32 × 32 bits)
...


PC
Address
Instruction
Instruction
memory
Data
Register #
Registers
Register #
Register #
ALU
Address
Data
memory
Data


Instruction: add $t0, $t1, $t2 (R-format)
rs (5 bits) ($t1)
rt (5 bits) ($t2)
rd (5 bits) ($t0)
opcode (6 bits)
funct (6 bits)
Control (decoder)
ALU control (ADD)
Read register #1 (rs = $t1)
Read register #2 (rt = $t2)
Write register # (rd = $t0)
1 Read $t1 = 10 (from register file)
Register File
Registers (32 × 32 bits)
Read data 1 (from $t1)
Read data 2 (from $t2)
10
32 bits
20
32 bits
ALU (ADD)
3 ALU result 30 (32 bits)
30
32 bits
2 Read $t2 = 20
4 Write back to register $t0 (rd = $t0)
30
32 bits
PC
Address
Instruction
Instruction memory
Data
Register #
Registers
Register #
Register #
ALU
Address
Data memory
Data

CPU Overview



CPU Overview



add $t0, $t1, $t2

The execution can be summarized in five steps:

1. Fetch the instruction
2. Decode the instruction
3. Read the registers
4. Execute and write back
5. Update the PC

If the instruction is changed to sub $t0, $t1, $t2, which steps remain the same, and which step changes?

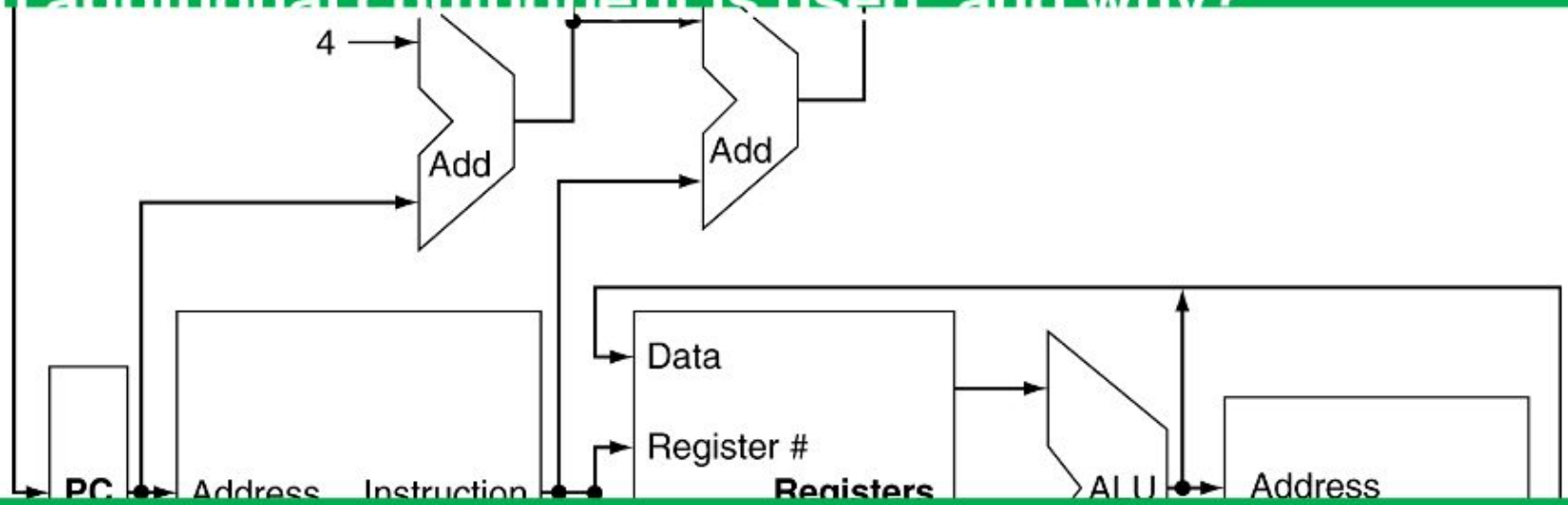
A2.

Step 4 changes: ALU operation: ADD → SUB. The processor

Q3.

(a) If the instruction is changed to lw $t0, 8($t1), which additional component is used, and why?

(b) If the instruction is changed to a branch instruction, which additional component is used, and why?



A3.

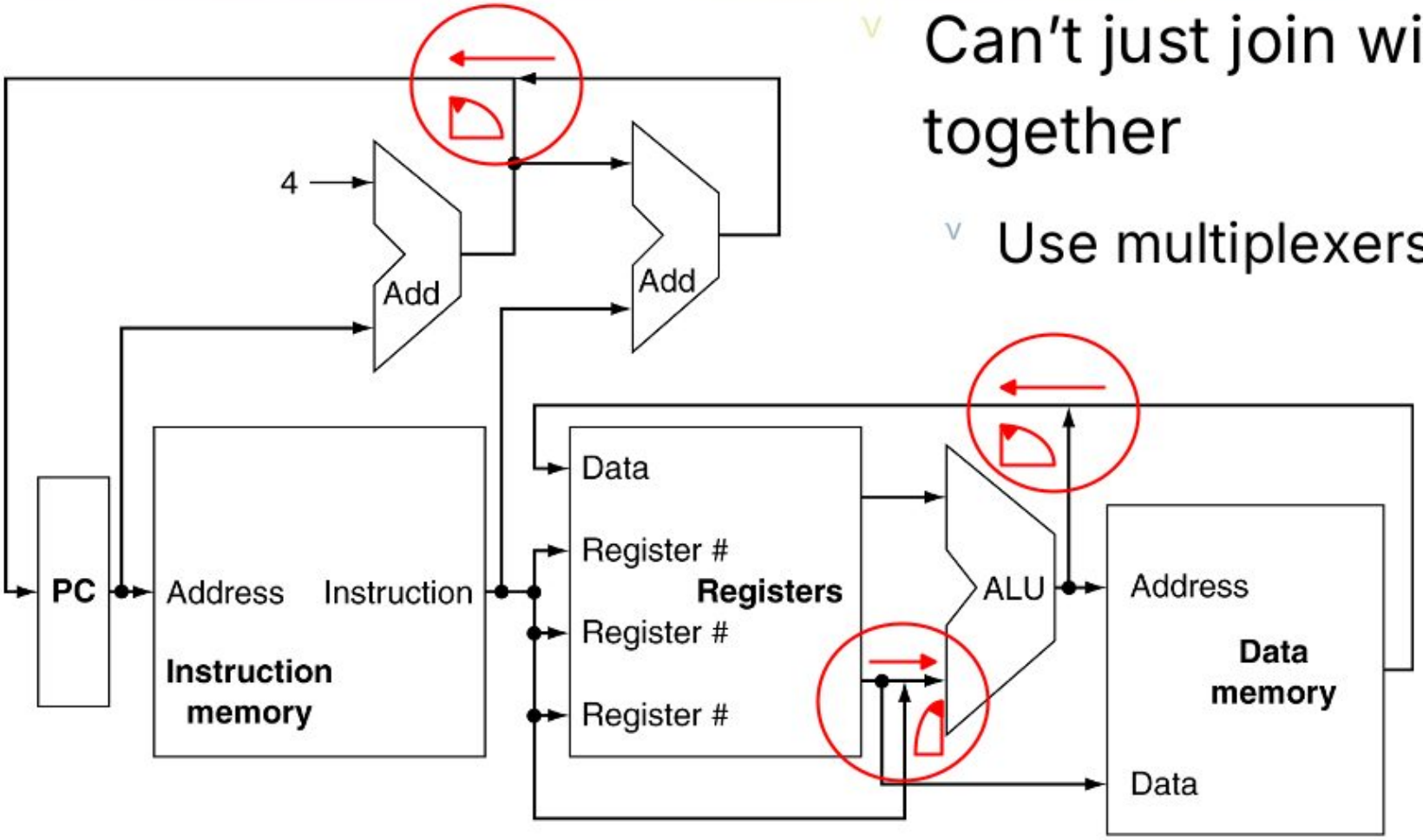
(a) Data Memory is additionally used.

The ALU calculates the memory address, and Data Memory provides the data to be loaded into $t0.

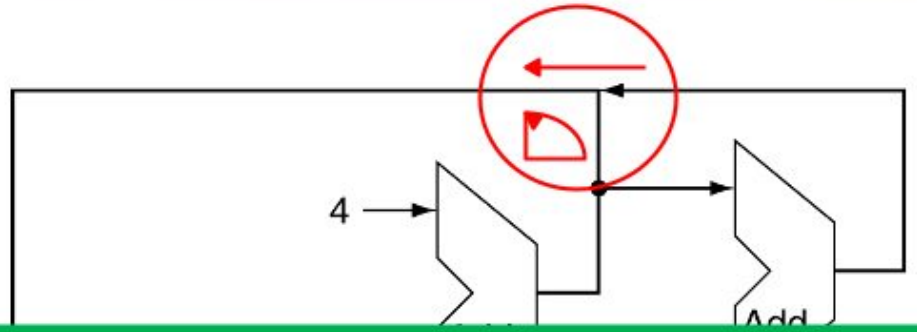
(b) The second Add unit calculates the branch target address using PC + 4 and the branch offset.

Multiplexers

- Can't just join wires together
  - Use multiplexers



Multiplexers



- Can't just join wires together
  - Use multiplexers

(a) What inputs does a MUX need to perform this selection?

(b) How does the MUX know which input should be selected?

A4.

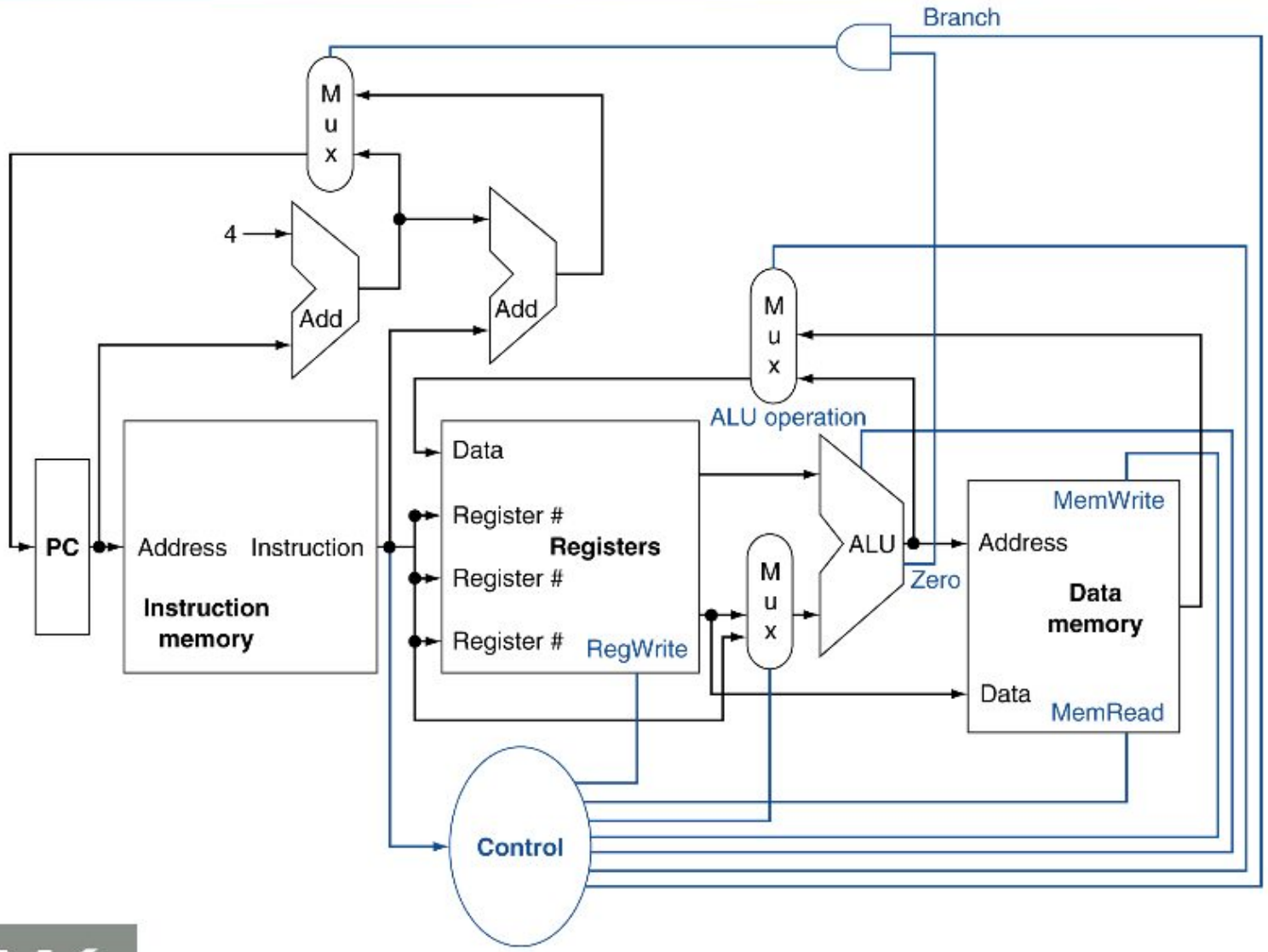
A MUX needs:

Multiple data inputs

A control (select) signal

(b) The control signal determines which data input is passed to the output

Control


Branch
M
u
x
4
Add
Add
M
u
x
ALU operation
PC
Address
Instruction
Instruction
memory
Data
Register #
Registers
Register #
Register #
RegWrite
M
u
x
ALU
Zero
MemWrite
Address
Data
memory
Data
MemRead
Control

Control

Fetch the instruction

Decode the instruction

Read the registers

Execute and write back

Update the PC

Which of these steps require control signals to determine what the processor should do?

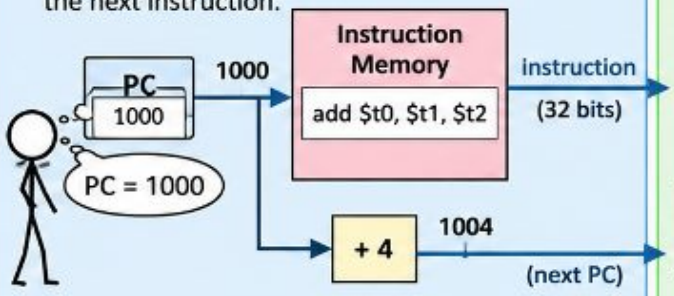
Step	Control Decision
1. Fetch	Fetch the instruction specified by the PC
2. Decode	Determine the required operation
3. Read Registers	Determine which registers are used
4. Execute & Write Back	Select ALU operation and enable RegWrite, etc.
5. Update PC	Select the next PC: PC + 4 or branch target

Control

CPU Overview	Function	New Signals	Function
PC	Stores the address of the current instruction	RegWrite	Enables writing a result to the Register File
Instruction Memory	Fetches the instruction using the PC address	ALU operation	Selects the arithmetic or logic operation for the ALU
Registers	Reads source operands and stores results	MemRead	Enables reading data from Data Memory
ALU	Performs arithmetic / logic operations and address calculation	MemWrite	Enables writing data to Data Memory
Data Memory	Reads or writes data for load/store instructions	Branch	Controls whether the branch path is used
Add / PC + 4	Calculates the next sequential instruction address	Zero	Indicates whether the ALU result is zero; used for branch decisions

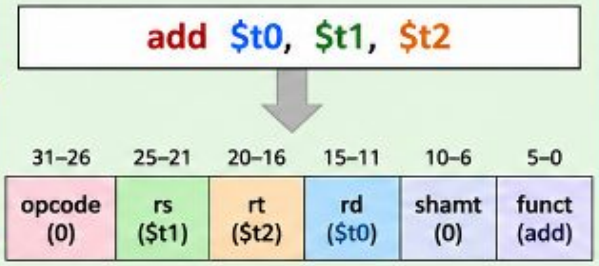
1 Fetch Instruction

- Use PC to fetch the instruction from Instruction Memory.
- Compute PC + 4 for the next instruction.



2 Decode Instruction

- Decode the fields of the instruction.
- Identify source and destination registers and the **operation (funct)**.



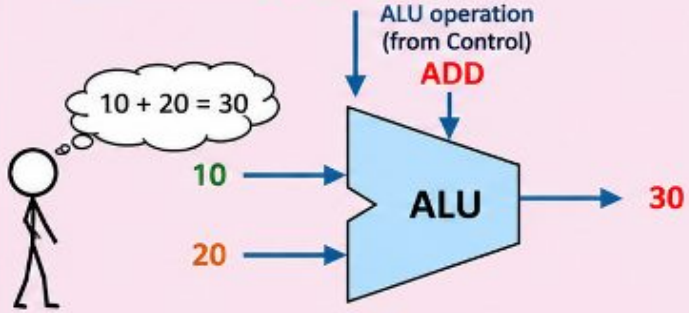
3 Read Registers

- Read the values of the source registers from the Register File.
- For add $t0, $t1, $t2, read $t1 and $t2.



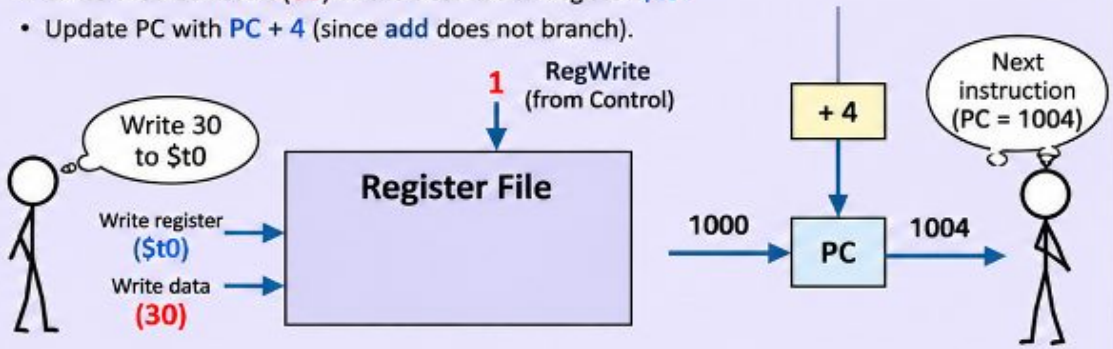
4 Execute (ALU Operation)

- The ALU performs the operation specified by the instruction.
- For **add**, compute: 10 + 20 = 30.

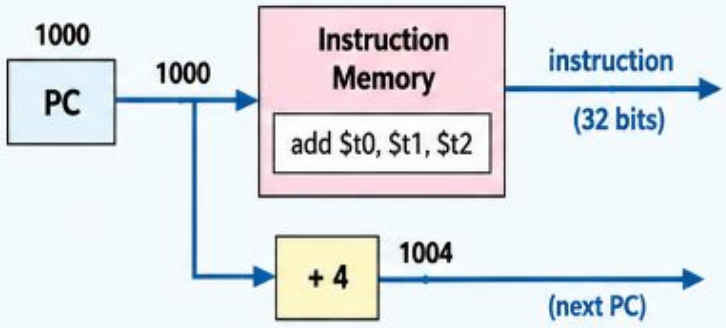


5 Write Back and Update PC

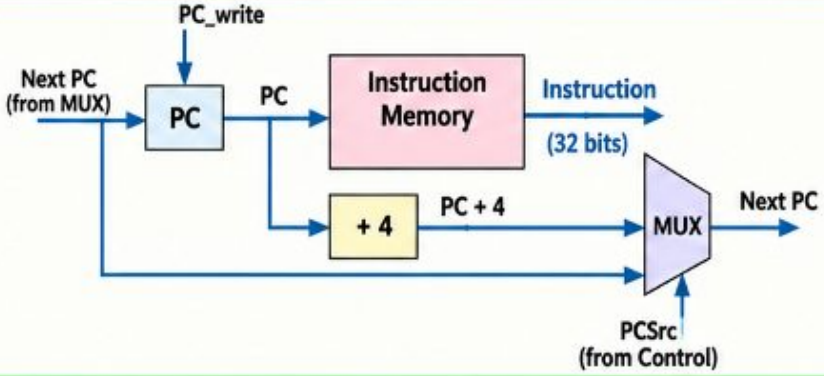
- Write the ALU result (30) to the destination register $t0.
- Update PC with PC + 4 (since add does not branch).



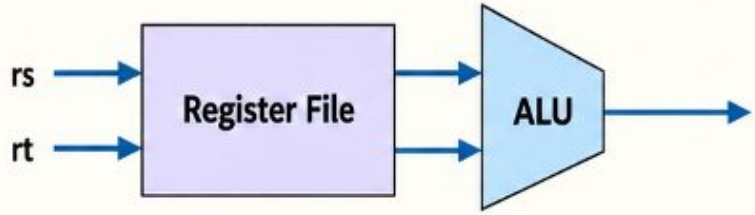
CPU Overview (3-1) — Instruction Fetch



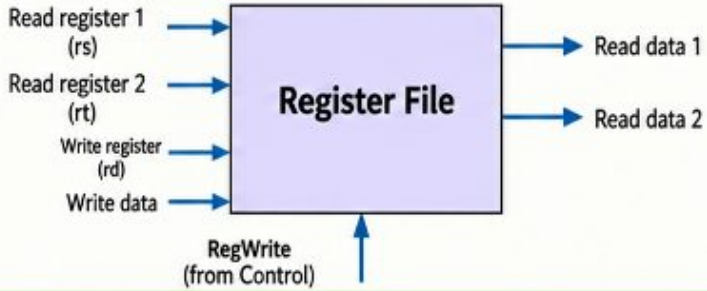
Later: Instruction Fetch Datapath



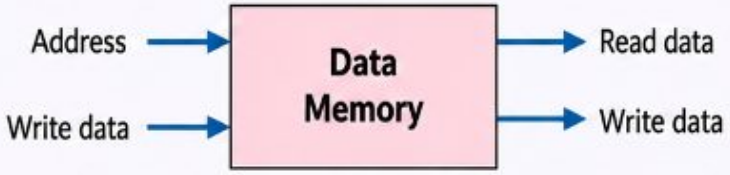
CPU Overview (3-1) — Register Access



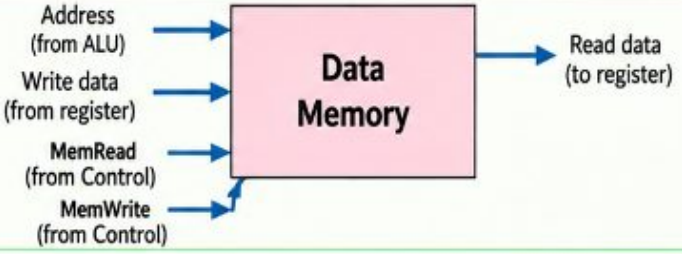
Later: Register File in Detail



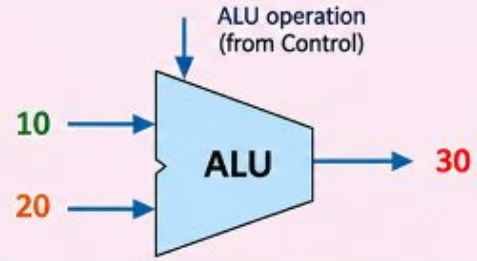
CPU Overview (3-1) — Data Memory



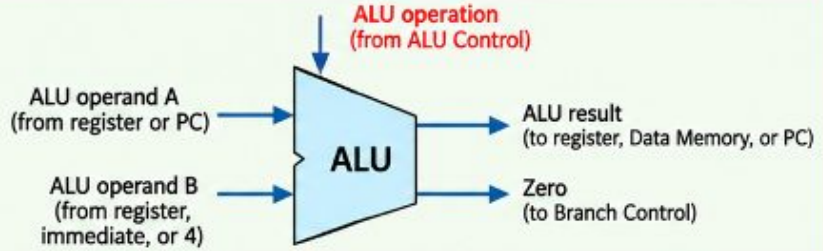
Later: Data Memory in Detail



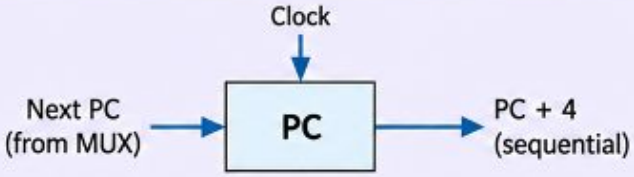
CPU Overview (3-1) — ALU Operation



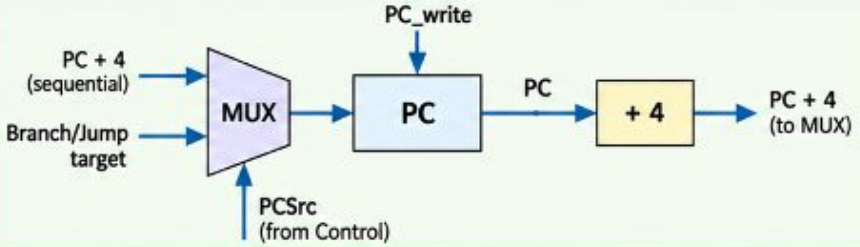
Later: ALU in Detail



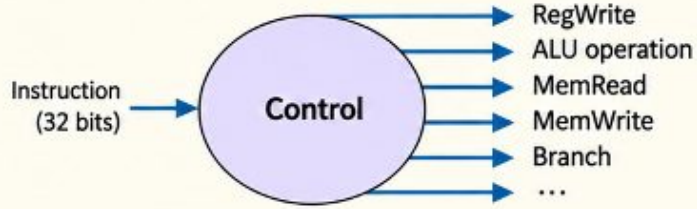
CPU Overview (3-1) — PC Update



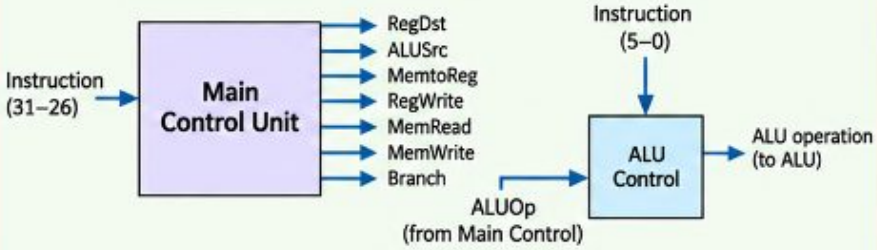
Later: PC Update in Detail



CPU Overview (3-1) — Control



Later: Main Control Unit in Detail



Section 3-1 Quick Questions

Q1. Arrange the major instruction-execution operations in the correct order.

Q2. For add → sub, which of the five execution steps remain the same, and which step changes?

Q3. Compared with add, what additional components are used by lw and branch instructions?

Q4. What inputs does a multiplexer need, and how does it know which input to select?

Q5. In the five execution steps, where is control needed? Give examples of control decisions.

Exercise 1 | Trace an Instruction

Consider: sub $t0, $t1, $t2

Assume $t1 = 25, $t2 = 9, and PC = 0x00400020.

(a) Trace the major execution steps and identify the CPU component used.

(b) Determine the value written to $t0.

(c) Determine the next PC value.

(a) **Answer**

PC → Instruction Memory → Register File → ALU → Register File → PC update

Register File reads $t1 = 25 and $t2 = 9.

ALU performs subtraction and the result is written back to $t0.

(b) $t0 = 25 − 9 = 16

(c) Next PC = 0x00400020 + 4 = 0x00400024

Key Point: Trace an instruction by following data through the major CPU components.

Exercise 2 | Compare Instruction Paths

Consider three instructions:

add $t0, $t1, $t2 lw $t0, 8($t1) beq $t0, $t1, Label

(a) For each instruction, determine whether it uses the Register File, ALU, Data Memory, and branch-target Add unit.

(b) Which components are shared by all three instructions?

Answer

Component	add	lw	beq
Register File	Yes	Yes	Yes
ALU	Yes	Yes	Yes
Data Memory	No	Yes	No
Branch-target Add	No	No	Yes

(b) The Register File and ALU are shared by all three instructions.

Key Point: Different instruction types reuse common hardware but activate different components.

Exercise 3 | Datapath Selection and Control

A processor shares one datapath among different instructions:

add $t0, $t1, $t2 lw $t0, 8($t1) beq $t0, $t1, Label

(a) Identify the main ALU role for each instruction.

(b) Determine whether Data Memory and the branch-target Add unit are needed.

Answer

(a) add: add register values

lw: calculate address

beq: compare registers

(b) add: neither | lw: Data Memory | beq: branch-target Add

Key Point: MUXes and control signals let different instructions share the same datapath.